

Class 6: Aggregation Operators

Execute Aggregation operations (\$avg, \$min,\$max, \$push, \$addToSet etc.). students encourage to execute several queries to demonstrate various aggregation operators)

What is aggregation?

Aggregation in MongoDB is a powerful framework for data aggregation operations, used to process data records and return computed results. It can be compared to the SQL GROUP BY clause but offers much more flexibility and capability. The aggregation framework operates through a pipeline approach, where multiple stages are used to transform and filter documents.

Grouping :Aggregation enables you to group documents based on shared characteristics. You can group by one or more fields within your documents.

This grouping helps you analyze data for specific categories or segments within your collection.

Manipulation:Once documents are grouped, you can perform various operations on the grouped data. These operations can include:

- Calculating averages, minimums, maximums, and other statistical measures.
- Counting the number of documents within each group.
- Creating new fields based on calculations or transformations of existing fields.

Aggregation Framework:MongoDB provides an Aggregation Framework specifically designed for performing these aggregation operations.

The framework uses a series of stages, each performing a specific transformation on the data as it flows through the pipeline.

Common stages include \$group, \$match, \$project, \$unwind, and various other operators for calculations and data manipulation.

Benefits of Aggregation:

- Aggregation offers several advantages, including:
- Efficient data exploration and analysis without needing to write complex queries.

- Ability to perform calculations and transformations on the server-side, reducing the workload on your application.
- Creation of new derived data fields for further analysis.

Syntax:

db.collection.aggregate(<AGGREGATE OPERATION>)

The code `db.collection.aggregate(<AGGREGATE OPERATION>)` represents using the Aggregation Framework in MongoDB to process and transform data stored in a collection.

Types:

Expression Type	Description	Syntax
Accumulators	Perform calculations on entire groups of documents	
* \$sum	Calculates the sum of all values in a numeric field within a group.	"\$fieldName": { \$sum: "\$fieldName" }
* \$avg	Calculates the average of all values in a numeric field within a group.	"\$fieldName": { \$avg: "\$fieldName" }
* \$min	Finds the minimum value in a field within a group.	"\$fieldName": { \$min: "\$fieldName" }
* \$max	Finds the maximum value in a field within a group.	"\$fieldName": { \$max: "\$fieldName" }
* \$push	Creates an array containing all unique or duplicate values from a field	"\$arrayName": { \$push: "\$fieldName" }
* \$addToSet	Creates an array containing only unique values from a field within a group.	"\$arrayName": { \$addToSet: "\$fieldName" }
* \$first	Returns the first value in a field within a group (or entire collection).	"\$fieldName": { \$first: "\$fieldName" }
* \$last	Returns the last value in a field within a group (or entire collection).	"\$fieldName": { \$last: "\$fieldName" }

Average GPA of All Students

```
db.students.aggregate([{$group:{_id:null, averageGPA: {$avg:"$gpa"}}}]);
```

The provided MongoDB aggregation query calculates the average GPA of all students in the `students` collection.

- ❑ **db.students.aggregate(...):** This line initiates the aggregation pipeline on the "students" collection within the database. The aggregation framework allows you to perform various operations to transform and group data.
- ❑ **[\$group:{...}]:** This defines an array containing a single aggregation stage. Each stage in the pipeline manipulates the data passed from the previous stage.
- ❑ **_id: null:** This specifies that we don't want to group documents based on any specific field. Setting `_id` to `null` creates a single group containing all documents from the "students" collection.
- ❑ **averageGPA: {\$avg: "\$gpa"}:** This is the key part where the average GPA is calculated.

Output:

```
[ { _id: null, averageGPA: 2.98556 } ]
db> |
```

Explanation:

- ❑ **_id: null:** This confirms that the aggregation didn't group the data based on any field (`_id` is set to `null`). It calculated the average GPA for the entire collection.
- ❑ **averageGPA: 2.98556:** This field stores the calculated average GPA, which is approximately 2.9856.

Minimum and Maximum Age:

```
db.students.aggregate([{$group:{_id: null, minAge: {$min:"$age"},maxAge:{$max:"$age"}}}]);
```

This code snippet again utilizes the aggregation framework in MongoDB to find the minimum and maximum age of students in the "students" collection.

- `db.students.aggregate()` is the entry point for the aggregation framework on the `students` collection.
- `{ $group: { _id: null, minAge: { $min: "$age" }, maxAge: { $max: "$age" } } }` is a grouping stage in the aggregation pipeline.
- `_id: null:`
 - This specifies that all documents will be grouped into a single group. The `null` value for `_id` means that there is no specific grouping key, and all documents are considered as a single group.
- `minAge: { $min: "$age" }:`
 - This creates a new field named `minAge` in the output document.
 - `$min: "$age"` calculates the minimum value of the `age` field across all documents in the `students` collection.
- `maxAge: { $max: "$age" }:`
 - This creates a new field named `maxAge` in the output document.
 - `$max: "$age"` calculates the maximum value of the `age` field across all documents in the `students` collection.

The entire code calculates two values:

- The minimum age (`minAge`) of any student in the collection.
- The maximum age (`maxAge`) of any student in the collection.

```
[ { _id: null, minAge: 18, maxAge: 25 } ]
```

Explanation:

- Similar to the previous example, it uses `$group` to group all documents.
- `minAge`: Uses the `$min` operator to find the minimum value in the "age" field.
- `maxAge`: Uses the `$max` operator to find the maximum value in the "age" field.

How to get Average GPA for all home cities?

```
db.students.aggregate([
```

```
{ $group: { _id: "home_city", averageGPA: { $avg: "$gpa" } } }
```

```
])
```

This code provides the aggregation framework in MongoDB to calculate the average GPA for students grouped by their "home_city".

The entire code calculates the average GPA for students in each city present in the "home_city" field. The output will be a collection of documents, one for each unique city. Each document will have two fields:

- `_id`: This field will contain the city name (the value from the "home_city" field that determines the group).
- `averageGPA`: This field will store the calculated average GPA for all students residing in that city.

```
db> db.students.aggregate([
...   { $group: { _id: "$home_city", averageGPA: { $avg: "$gpa" } } }
...   ]);
[
  { _id: 'City 8', averageGPA: 3.11741935483871 },
  { _id: 'City 7', averageGPA: 2.847931034482759 },
  { _id: 'City 10', averageGPA: 2.935227272727273 },
  { _id: 'City 9', averageGPA: 3.1174358974358976 },
  { _id: 'City 2', averageGPA: 3.0196969696969697 },
  { _id: 'City 3', averageGPA: 3.0100000000000002 },
  { _id: 'City 6', averageGPA: 2.8969444444444448 },
  { _id: null, averageGPA: 2.9784313725490197 },
  { _id: 'City 4', averageGPA: 2.8251851851851852 },
  { _id: 'City 1', averageGPA: 3.003823529411765 },
  { _id: 'City 5', averageGPA: 3.0607499999999996 }
]
```

This provides the aggregation pipeline successfully identified unique courses across all candidate documents in your "candidates" collection.

- **_id: null:** This confirms that the aggregation didn't group the data based on any specific field because `_id` is set to `null`. It calculated unique courses for the entire collection, considering all courses mentioned by all candidates.
- **uniqueCourses:** This field stores an array containing the unique course names found in the "courses" field of each candidate document. The list you might see in the output would include all the unique course names, not just the three examples ("Math", "Physics", "Chemistry").

Pushing All Courses into a Single Array

```
db.Students.aggregate([
```

```
{ $project: { _id: 0, allCourses : { $push: "$courses" } } }]);
```

- **Aggregation Framework:**

- `db.students.aggregate()` is the entry point for the aggregation framework on the `students` collection.

- **\$project Stage:**

- `{ $project: { _id: 0, allCourses: { $push: "$courses" } } }` is a projection stage in the aggregation pipeline.
- `_id: 0:`
 - This excludes the `_id` field from the output documents.
- `allCourses: { $push: "$courses" }:`
 - The `$push` operator is used to append values to an array in the resulting documents.
 - `"$courses"` specifies the field whose values will be pushed into the `allCourses` array.

The entire code snippet transforms the student documents by:

- Removing the original `_id` field (if not required in the output).
- Adding a new field named "allCourses".

- Populating the "allCourses" field with an array containing all the courses listed in the student's original "courses" field (assuming it's an array).

Explanation:

- **\$project**: Transforms the input documents.
 - **_id: 0**: Excludes the **_id** field from the output documents.
 - **All Courses**: Uses the **\$push** operator to create an array. It pushes all elements from the "courses" field of each student document into the **allCourses** array.

Result:

This will return a list of documents, each with an **allCourses** array containing all unique courses offered (assuming courses might be duplicated across students).

BUT:

```
db> db.students.aggregate([
...   { $project: { _id: 0, allCourses: { $push: "$courses" } } }
... ]);
MongoServerError[Location31325]: Invalid $project :: caused by :: Unknown expression $push
db> |
```

This is because our Array is incorrect:)

Collect Unique Courses Offered (Using \$addToSet);

```
db.candidates.aggregate([
  { $unwind: "$courses" },
  { $group: { _id: null, uniqueCourses: { $addToSet: "$courses" } } }
]);
```

The provided MongoDB aggregation query extracts a unique list of courses from the candidates collection.

Aggregation Framework:

`db.students.aggregate()` is the entry point for the aggregation framework on the `students` collection.

\$group Stage: `{ $group: { _id: null, minAge: { $min: "$age" }, maxAge: { $max: "$age" } } }` is a grouping stage in the aggregation pipeline.

Result:

The result of this aggregation query will be a single document containing the minimum and maximum ages of all students:

What does it do?

```
db> db.candidates.aggregate([
...   { $unwind: "$courses" }, // Deconstruct courses array
...   { $group: { _id: null, uniqueCourses: { $addToSet: "$courses" } } }
que courses
... ]);
[
  {
    _id: null,
    uniqueCourses: [
      'Sociology',
      'Literature',
      'Ecology',
      'Physics',
      'Mathematics',
      'Marine Science',
      'Artificial Intelligence',
      'Art History',
      'Creative Writing',
      'Robotics',
      'Environmental Science',
      'Biology',
      'Statistics',
      'Music History',
      'Philosophy',
      'Film Studies',
      'Engineering',
      'Computer Science',
      'English',
      'Psychology',
      'Chemistry',
      'Political Science'
    ]
  }
]
```

- `_id: null`: This confirms that the aggregation didn't group the data based on any field (`_id` is set to `null`). It calculated unique courses across the entire collection.
- `uniqueCourses`: This field stores an array containing all the unique course names found in the "courses" field of each candidate document. The document snippet you provided truncates the list after a few elements (`[...]`).